OPTIMIZATION TECHNIQUES

Assignment 8: Optimize Anything

Technical Report

Ten-track optimization experiments using AI-assisted development

Name: Sofia Abidi

Reg. No: 241071072

B. Tech. Computer Engineering

https://github.com/sofiaabidi/optimization-labRepository: https://github.com/sofiaabidi/optimization-lab

Environment: Ubuntu 26.04 under WSL2; Python 3.11; CPU-based numerical and quantum simulation; QEMU for xv6.

Report prepared from recorded repository results and the supplied assignment specification, October 2026. Student details can be added on the title page before submission.

Abstract

This report documents ten optimization experiments spanning continuous and discrete methods, quantum circuits, parallel and distributed systems, an applied scheduling case study, compiler transformations, an operating-system kernel, database indexing, quantum machine learning, and software testing. Each study specifies the objective and constraints, establishes a baseline, applies suitable alternatives, and checks whether the resulting output remains valid. Some methods **improve the objective substantially**: a linear-programming model reduces the cost of a small campus charging schedule by **24.1%**, and storage-budgeted indexing lowers the weighted **latency of the measured SQLite workload by 88.9%**. Other experiments show costs or limitations: priority scheduling improves selected response times but can reduce throughput; quantum simulations use fewer oracle queries in a Grover example but run much slower than classical search; a dynamic-programming compiler pass order does not outperform a fixed order on held-out programs. The results support the central conclusion that optimization methods must be evaluated against their objectives, constraints and actual execution conditions, rather than judged by algorithm names alone.

1. Introduction

The assignment asks us to identify an inefficient process, express it mathematically, choose an optimization method, and demonstrate whether the result improves on the original process. It

also requires ten focused experiment tracks, A through J. We organized them in a single reproducible repository, using common command-line and result-recording utilities where appropriate. The work is deliberately experimental: it is not a claim that every benchmark is representative of an industrial deployment.

A recurring principle guided the experiments: **the most sophisticated algorithm is not necessarily the best one**. Greedy methods sometimes matched an exact integer program; parallel execution sometimes lost to communication overhead; and faster response time did not necessarily mean higher throughput. These differences are as important as positive speedup figures.

2. Experimental setup and verification

The main development environment used **WSL2 Ubuntu, a Python 3.11 virtual environment managed with uv, and pinned packages including NumPy, SciPy/HiGHS, PyTorch and PennyLane**. SQLite was used for the database experiment. The xv6 track compiled a pinned upstream **RISC-V kernel and ran it under QEM**U. Most numerical experiments used deterministic synthetic data, consistent seeds and local CPU execution. Quantum circuit experiments used local simulators rather than quantum hardware.

Each track records an objective, inputs, raw measurements, a comparison against an appropriate baseline, and correctness checks. Numerical derivatives were checked against independent oracles, combinatorial problems against exhaustive solutions where tractable, compiler passes against unoptimized execution, SQL selections against identical query results, and the modified xv6 kernel against its real boot and usertests. The repository reports **56 passing fast tests** and cross-track artifact consistency following the final work-stealing correction. These are the recorded project checks, not tests rerun while preparing this report.

Track	Experiment	Main validity check
A	Continuous optimizers	Analytic optimum; finite differences; autograd
B	Knapsack and QUBO/Ising	Brute-force enumeration
C	QAOA/VQE/QNN	Hamiltonian diagonal; classical oracle
D	Parallel/distributed optimization	Exact-once task execution; numerical equality
E	Campus EV charging	Independent schedule feasibility
F	Compiler passes	AST/IR/VM output equivalence
G	xv6 scheduling	QEMU boot and xv6 usertests
H	SQLite indexing	Query checksum equality
I	Quantum algorithms and QML	Known circuit outputs; held-out data
J	Test selection	Real test runs and isolated mutations

Quick profiles were executed for all tracks. The repository separately records full profiles for A–F; full-scale G–J profiles were not run. Results quoted below are identified by their experiment context, rather than combining incompatible benchmark settings.

3. Track A — Continuous optimization

3.1 Problem and methods

A regularized linear regression problem was used as the main common objective. For a weight vector w, feature matrix X, target vector y and regularization coefficient $\lambda = 0.1$, the loss was $L(w) = (1/2n)||Xw - y||^2 + (\lambda/2)||w||^2$. The small fixed dataset had 96 observations and three parameters. Because its minimum Hessian eigenvalue was positive (0.89933), the objective was strongly convex. Gradient descent was the reference method.

We compared full-batch gradient descent, stochastic and mini-batch variants, Momentum, Nesterov acceleration, AdaGrad, RMSProp, Adam and Newton. Gauss–Newton was tested on an appropriate nonlinear exponential least-squares problem rather than forced into the ridge-regression ranking. Additional functions illustrated the difference between strongly convex and merely convex optimization, local and global minima, saddle points and discrete versus continuous decisions.

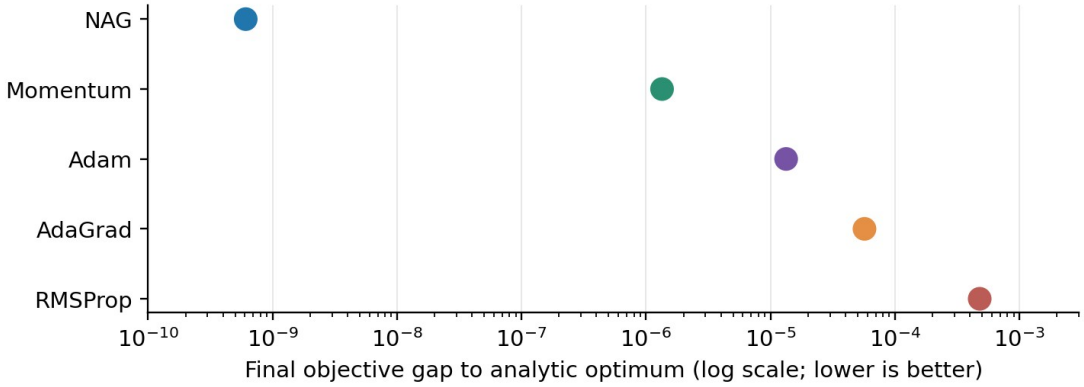


Figure 1. Objective gaps for selected first-order methods on the fixed ridge-regression case. Note the logarithmic scale.

3.2 Results and interpretation

Method	Quick final objective	Observation
GD	0.326068495	Reached analytic minimum
NAG	0.326068495	Near-zero objective gap
Adam	0.326081848	Small residual gap

RMSProp	0.326547493	Larger residual gap at fixed settings
SGD, mean of 3	0.334525218	More variable
Newton	0.326068495	3 updates on small 3-D case

The analytic minimum was 0.**3260684949**. The hand-derived gradient agreed with PyTorch automatic differentiation to 4.44×10^{-16} and with finite differences to approximately 1.02×10^{-10}. The Hessian-vector product also matched its numerical check. Newton appeared fast on this three-parameter example, but its cost grows with forming and solving a curvature system. The learning-rate sweep demonstrated slow progress for an undersized step and divergence for an excessive one. These experiments highlight tuning and problem geometry rather than universal optimizer rankings.

https://github.com/sofiaabidi/optimization-lab/tree/main/tracks/a_continuoushttps://github.com/sofiaabidi/optimization-lab/tree/main/results/aSource and reproducibility: https://github.com/sofiaabidi/optimization-lab/tree/main/tracks/a_continuous; numerical artifacts in https://github.com/sofiaabidi/optimization-lab/tree/main/results/a.

4. Track B — Discrete optimization

The primary combinatorial problem was **0/1 knapsack**. Each item had a binary inclusion decision; the objective was to maximize total value without exceeding capacity 33. We compared **value-to-weight ratio greedy selection, capacity dynamic programming, branch-and-bound and SciPy/HiGHS mixed-integer optimization**. Enumeration of all 4,096 possible selections provided an independent optimum.

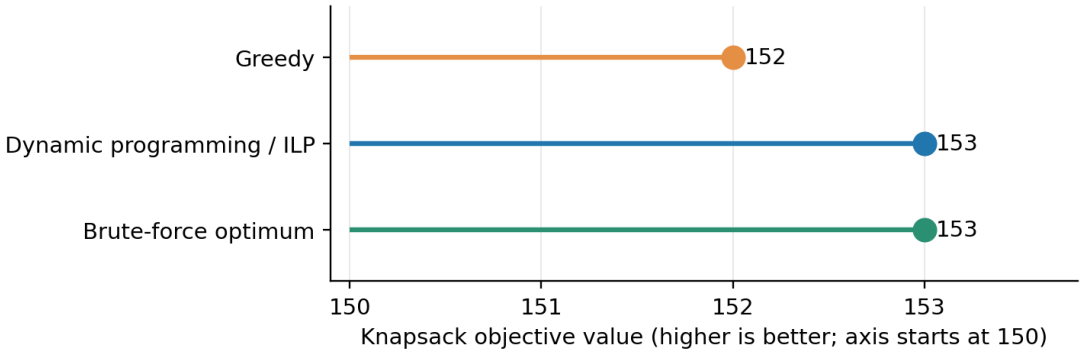


Figure 2. Knapsack objective values. The greedy heuristic missed the optimum by one unit; the axis is deliberately restricted to show this difference.

Method	**Objective value**	**Feasible weight**
Greedy	152	30
Dynamic programming	153	32

Branch-and-bound	153	32
MILP	153	32
Brute-force optimum	153	32

The greedy algorithm missed the optimum by one value unit. The three exact approaches agreed with enumeration. A separate four-node weighted Max-Cut instance was converted into a binary quadratic unconstrained optimization (QUBO) objective and an Ising form using binary-to-spin conversion. All 16 assignments gave matching energies; the maximum cut was 6.5. A constrained selection example with a squared penalty showed that too-small penalty weights admit infeasible selections, while a sufficiently large weight restores feasibility. The Max-Cut formulation was reused in Track C to maintain a consistent quantum-classical comparison.

Source and reproducibility: https://github.com/sofiaabidi/optimization-lab/tree/main/tracks/b_discrete; numerical artifacts in https://github.com/sofiaabidi/optimization-lab/tree/main/results/b.

5. Track C — Quantum optimization

The first experiment applied a two-layer Quantum Approximate Optimization Algorithm (QAOA) circuit to Track B's four-node Max-Cut problem. Its cost Hamiltonian used the verified Ising representation. The optimized simulator expectation was a cut value of 6.142389, compared with the exact classical maximum of 6.5. The circuit assigned probability 0.490765 to optimal bitstrings. It was therefore a useful approximation experiment, not evidence of quantum superiority.

Finite-shot estimates were repeated at 100, 500 and 2,000 measurements, demonstrating the decrease in sampling uncertainty. A synthetic depolarizing channel decreased expected cut from 6.142389 without noise to 5.721571 at the strongest displayed noise setting. A separate two-qubit VQE problem compared parameter-shift gradients, simultaneous perturbation stochastic approximation (SPSA), and an approximate diagonal-metric quantum natural gradient. Energy errors were approximately 1.31×10^{-5}, 7.71×10^{-4} and 1.36×10^{-5} respectively at similar circuit-evaluation budgets.

The experiment additionally included parameterized circuits, a small QNN and gradient-variance diagnostics for increasingly large circuit settings. Every circuit ran in PennyLane on a classical CPU simulator; the noise channel was not calibrated to any physical device. This distinction matters because simulator wall time is not quantum hardware performance.

Source and reproducibility: https://github.com/sofiaabidi/optimization-lab/tree/main/tracks/c_quantum_opt; numerical artifacts in https://github.com/sofiaabidi/optimization-lab/tree/main/results/c.

6. Track D — Submodular, parallel and distributed optimization

A maximum-coverage objective demonstrated submodularity: adding an additional set has diminishing benefit as more sets have already been chosen. For the tested cardinality limit of three, greedy selected three sets covering 10 elements and equalled the exhaustive optimum. The program checked the required diminishing-return and monotonicity assumptions, rather than assuming an approximation guarantee held automatically.

The parallel benchmark used real local worker processes and measured a median speedup of **2.656×**on four workers, or 66.4% efficiency. For skewed workloads, shared-queue dynamic scheduling reduced elapsed time from 0.2933 s to 0.1545 s. A later targeted audit also implemented actual per-worker deques: workers popped from their own right end, while idle workers stole tasks from the left end of another deque.

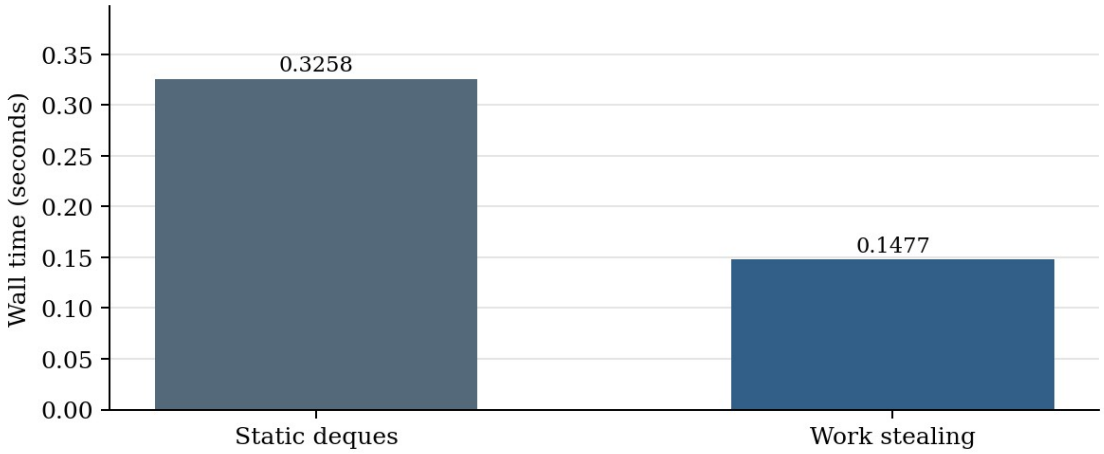


Figure 3. Per-worker deque scheduling on the same skewed workload; shorter elapsed time is better.

The deque demonstration reduced wall time from 0.3258 s to 0.1477 s, with three actual steals and exactly-once execution. It is an educational lock-based implementation, not a claim about a production lock-free scheduler. Local data and model parallelism were slower than serial execution because small tasks could not amortize process communication. Two-stage pipeline execution improved from 0.2682 s to 0.1839 s. Federated averaging (FedAvg) was simulated through sequential clients and compared against centralized learning; multi-task experiments tested MGDA and GradNorm against equal task weighting. The study also interpreted measured speedups with Amdahl's and Gustafson's laws. The communication payloads of FedAvg are calculated, not real network measurements.

Source and reproducibility: https://github.com/sofiaabidi/optimization-lab/tree/main/tracks/d_parallel_distributed; numerical artifacts in https://github.com/sofiaabidi/optimization-lab/tree/main/results/d.

7. Track E — Main case study: campus EV charging

7.1 Process and formulation

A campus charging operator needs to serve vehicles that arrive and depart at different times while electricity prices change between hourly slots. The unoptimized approach charges each vehicle as soon as capacity is available. This is feasible in the fixed case but uses expensive early slots unnecessarily. For each vehicle v and time slot t, the decision x(v,t) is delivered energy in kWh. We minimize the sum of price(t) × x(v,t), subject to each vehicle receiving exactly its requested energy, charging only while present, its power-rate limit, and the facility's slot capacity.

The fixed case uses four vehicles, six slots, 20 kWh of total demand and a 6 kW facility limit. We compared first-available charging, a cheapest-slot heuristic and a global linear program solved using HiGHS. Every schedule was checked independently for feasibility.

7.2 Results

Method	Feasible	Cost	Saving vs baseline	Median runtime
First available	Yes	4.48	—	0.121 ms
Cheapest slot	Yes	3.58	20.1%	0.118 ms
Exact LP	Yes	3.40	24.1%	1.190 ms

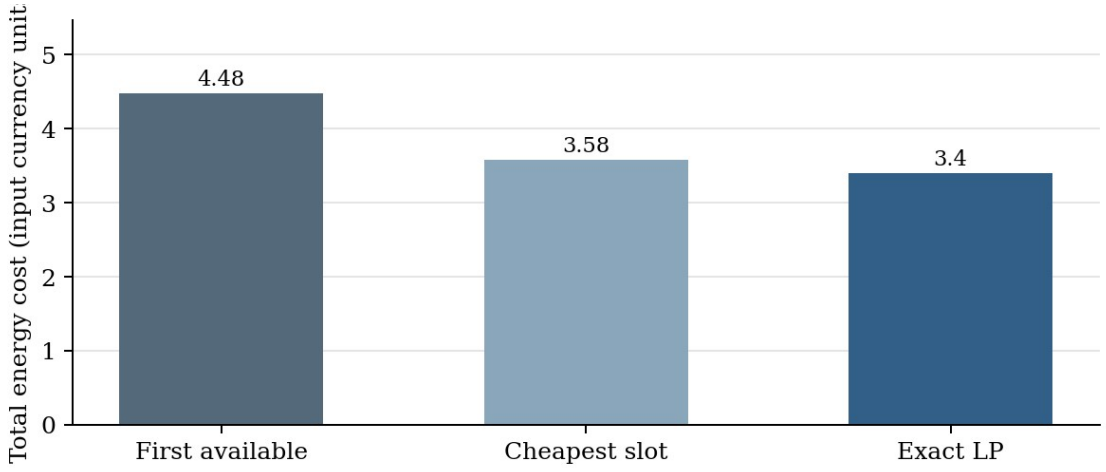


Figure 4. Charging cost for the fixed feasible case. The numerical units follow the input electricity tariff.

The LP reduced the cost by **24.1%**, but the facility peak remained 6 kW (100% utilization). This is not contradictory: we optimized energy cost, not maximum demand. The exact solver also carried more overhead than the heuristic on this small instance. At some larger fleet settings the

cheapest-slot heuristic failed to deliver all requested energy even though the LP found a feasible allocation. The deliberately infeasible one-slot vehicle request was correctly rejected, with nofalse low-cost optimum reported. At a less constrained capacity setting, heuristic and LP tied, showing a case where the exact solver provided no additional cost benefit.

The experiment demonstrates how the mathematical objective determines what improves and what does not. It is a simulated campus plan with known future arrival, demand and price information, not a deployed charging control system.

Source and reproducibility: https://github.com/sofiaabidi/optimization-lab/tree/main/tracks/e_ev_charging; numerical artifacts in https://github.com/sofiaabidi/optimization-lab/tree/main/results/e.

8. Track F — Compiler optimization

We implemented a small teaching language with assignments, integer arithmetic and print statements, including a lexer, recursive-descent parser, abstract syntax tree evaluator, three-address intermediate representation, bytecode generator and virtual machine. Four optimization passes were implemented: constant folding, constant propagation, dead-code elimination and common-subexpression elimination. Each transformation preserved the observable program output in the tested cases.

Pass ordering itself formed a discrete optimization problem. A subset dynamic program selected an order using a separate training program. While that choice decreased training-program code size from 61 to 56 bytes, its output was worse than a manually fixed pass order on the five held-out programs.

Held-out strategy	Instructions	Compiled size	Summed median VM time
Unoptimized	27	317 B	6.696 µs
Fixed four-pass	15	162 B	3.664 µs
DP-selected	17	191 B	3.729 µs

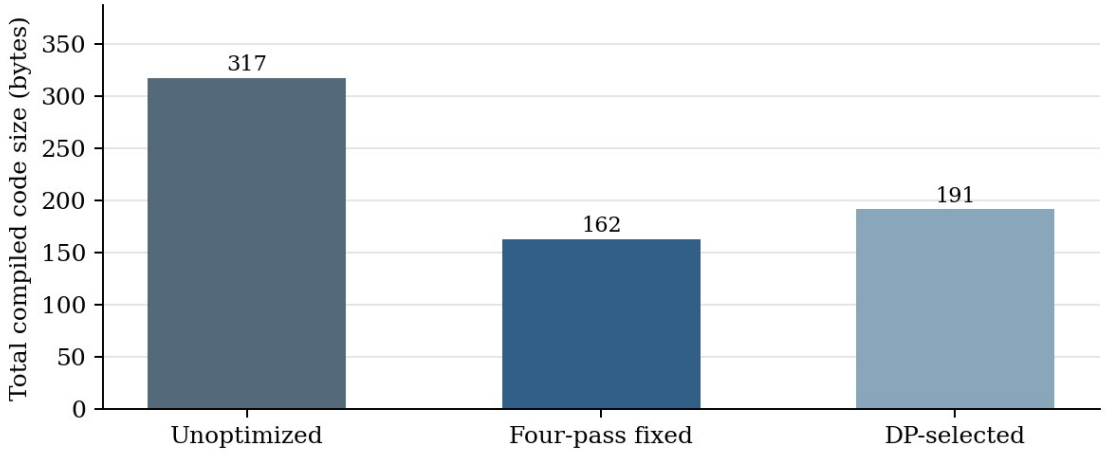


Figure 5. Combined compiled bytecode size across five held-out programs.

The fixed pass sequence cut instructions by 44.4% and code size by 48.9%, while compilation time rose modestly. Across fixture pipelines and generated programs, optimized outputs matched unoptimized AST, IR and VM execution. The unsuccessful generalization of the DP ordering is important: optimizing an intermediate decision on one training example does not guarantee the best choice on new programs.

Source and reproducibility: https://github.com/sofiaabidi/optimization-lab/tree/main/tracks/f_compiler; numerical artifacts in https://github.com/sofiaabidi/optimization-lab/tree/main/results/f.

9. Track G — xv6 operating-system optimization

This track modified a pinned checkout of the actual xv6-riscv teaching kernel rather than a scheduling simulator. We added strict-priority scheduling and a setpriority system call, plus a getpstats diagnostic interface to expose process dispatches and runtime ticks. The performance objective was to reduce response time for important processes; the statistics interface provides observability and is not itself a second performance optimization.

A clean upstream kernel and the modified kernel were both booted under QEMU. The repository retains patch files and actual terminal logs from three-hart boot and the full xv6 usertests, which finished with **ALL TESTS PASSED**. Scheduling benchmarks used one CPU to isolate policy effects and three repeats for CPU-intensive, mixed sleep/CPU and equal-priority contention workloads.

Metric	Upstream scheduler	Priority scheduler
Important CPU job response	6 ticks	1 tick
Important mixed-job response	18 ticks	6 ticks
CPU makespan	10 ticks	11 ticks
CPU throughput	33.0M work/tick	30.0M work/tick
Equal-priority fairness ratio	0.498	0.616

Response time improved by **83.3%** for the CPU job and **66.7%** for the mixed workload. However, CPU makespan increased, throughput fell 9.1%, and the scheduler incurs full process-table scan overhead. Strict priority may also starve low-priority processes because no aging mechanism was implemented. These results describe the intended trade-off of this specific scheduling policy and benchmark, not an overall improvement to every OS metric.

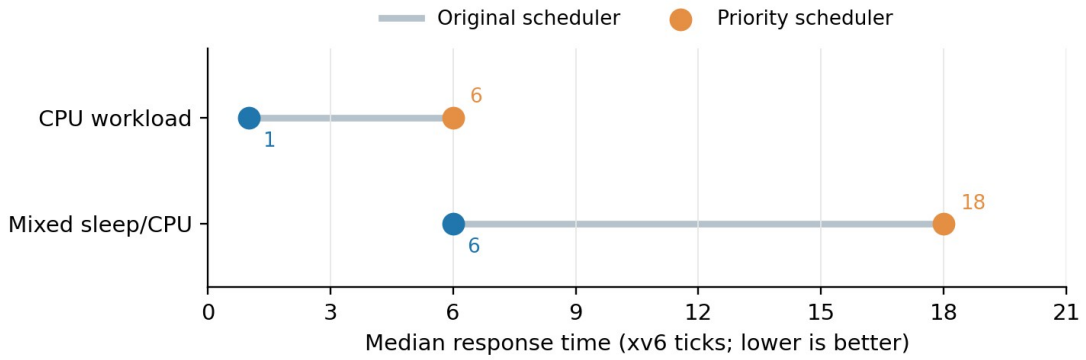


Figure 6. Paired response-time comparison for the xv6 scheduling experiment. Both workloads improve under priority scheduling.

Source and reproducibility: https://github.com/sofiaabidi/optimization-lab/tree/main/tracks/g_xv6; numerical artifacts in https://github.com/sofiaabidi/optimization-lab/tree/main/results/g.

10. Track H — SQLite database optimization

The database experiment optimized index selection subject to a 2.5 MB added-storage budget. A documented synthetic campus dataset contained 10,000 students, 240 courses and 60,000 enrollments. Five parameterized query templates formed a weighted workload. Six candidate indexes were considered, and every one of their 64 subsets was built and measured on a fresh copy of the database. The exact ILP selected among measured subsets, preserving index interactions rather than assuming each index benefit was independent.

Strategy	Added index storage	Weighted latency	Measured throughput
SQLite default	0	275.892 ms	405.7 queries/s
Conditional greedy	1,953,792 B	30.603 ms	2,691.3 queries/s
Measured-subset ILP	1,953,792 B	30.603 ms	2,691.3 queries/s

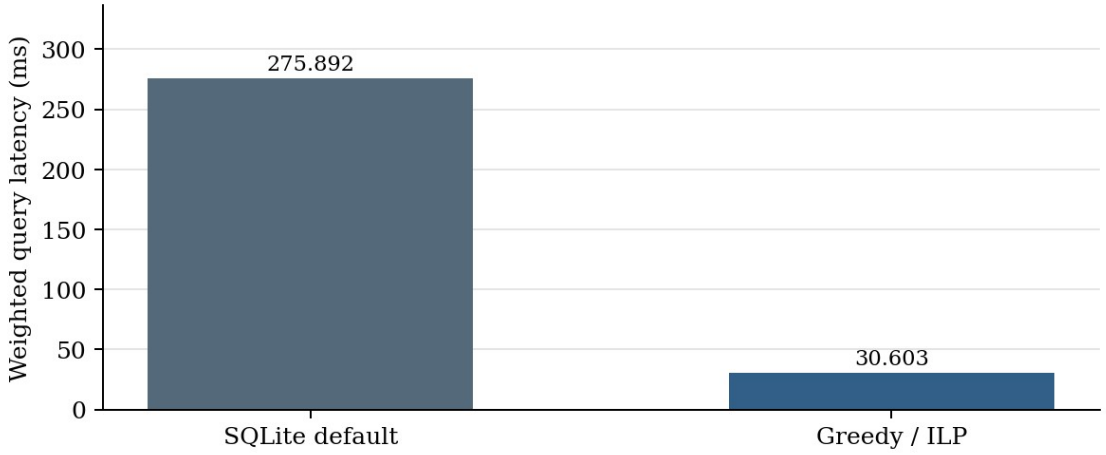


Figure 7. Weighted latency of the default and selected index configuration.

The selected three-index configuration reduced the weighted latency by **88.9%**and improved separately measured single-client throughput to 2,691.3 queries/s. Greedy and ILP found the same index subset, so the exact solver offered no further improvement for this dataset. Every query checksum matched across all 64 subsets, and all 15 selected query-case plans changed. Because the tests used warm cache, disposable databases and one local client, no production concurrency or cold-cache claim is made.

Source and reproducibility: https://github.com/sofiaabidi/optimization-lab/tree/main/tracks/h_dbms; numerical artifacts in https://github.com/sofiaabidi/optimization-lab/tree/main/results/h.

11. Track I — Quantum algorithms and machine learning

The simulator study assembled Deutsch–Jozsa, Bernstein–Vazirani, Grover, QFT, QPE and a compiled demonstration of Shor factoring 15. QFT was constructed from gates and checked by an inverse circuit. The Shor demonstration used a narrow precompiled modular-permutation circuit and classical GCD postprocessing; it should not be treated as a general factoring implementation. A classical simulated annealer was also included and labelled as classical.

In a search for item 5 out of eight possibilities, Grover needed two oracle queries, compared with six equality tests in the actual ascending linear search, but took 7.271 ms on the statevector simulator versus 0.0062 ms for Python search. The experiment illustrates a theoretical query advantage without a simulator runtime advantage.

Model	Held-out accuracy	Recorded time
Quantum fidelity kernel	81.25%	1.867 s
Classical RBF kernel	87.50%	0.000174 s
Variational quantum classifier	87.50%	20.301 s
Classical logistic comparator	93.75%	0.00728 s

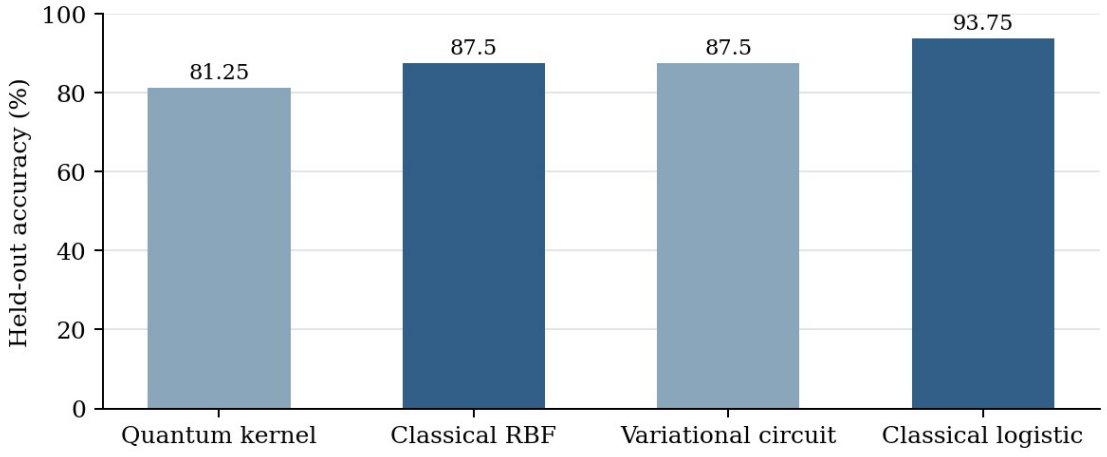


Figure 8. Held-out classification accuracy on the same small synthetic dataset (16 test observations).

The two quantum ML models were less accurate and required considerably more simulated computation than their matched classical baselines on the tested dataset. The finite-shot and depolarizing-noise studies likewise quantify sampling and noise sensitivity. These are not hardware measurements or evidence that quantum methods are universally worse; they show the limits of this particular small benchmark.

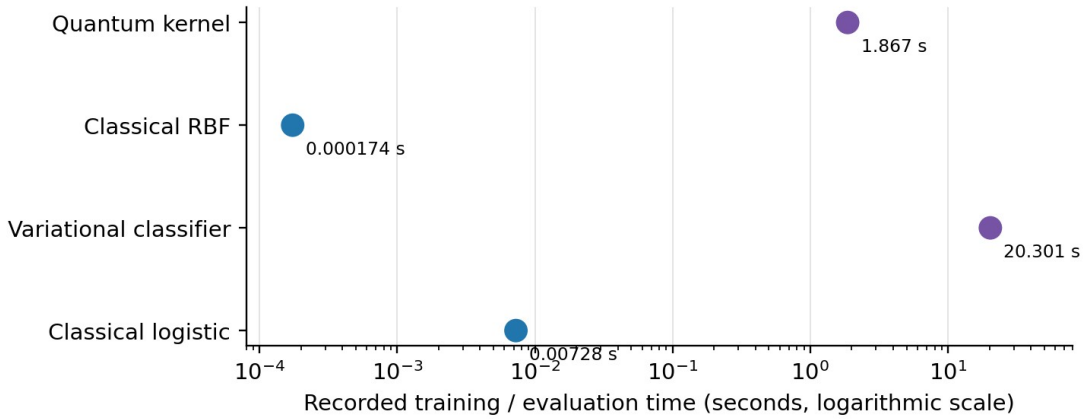


Figure 9. Recorded quantum-versus-classical model times. A logarithmic axis is necessary because the runtimes span several orders of magnitude.

Source and reproducibility: https://github.com/sofiaabidi/optimization-lab/tree/main/tracks/i_quantum_qml; numerical artifacts in https://github.com/sofiaabidi/optimization-lab/tree/main/results/i.

12. Track J — Software-engineering optimization

The final experiment used the project itself as a realistic codebase: 4,752 maintained pre-Track-J Python source lines and 54 real tests at the time of profiling. Its objective was to choose tests under a limited sum of measured test-phase durations so as to maximize weighted coverage of named features and observed fault detectors. We compared an alphabetical prefix baseline, greedy marginal coverage selection and an exact ILP. Coverage weights were assigned to test features and six deliberately injected faults. Mutations ran only in isolated temporary copies and do not represent genuine AI coding defects.

25% phase budget	Selected tests	Detected mutations	Actual selected-suite wall time
Alphabetical prefix	10	0/6	0.969 s
Greedy	51	6/6	3.793 s
Exact ILP	51	6/6	3.839 s

Both optimization methods detected all six controlled mutations within the declared test-phase budget and matched in objective value at every tested budget. However, startup and import costs meant that total selected-suite wall time was not reduced in direct proportion to the phase budget.

The normal CI suite continued to run all fast tests; selection was an experiment, not a change that weakened project verification.

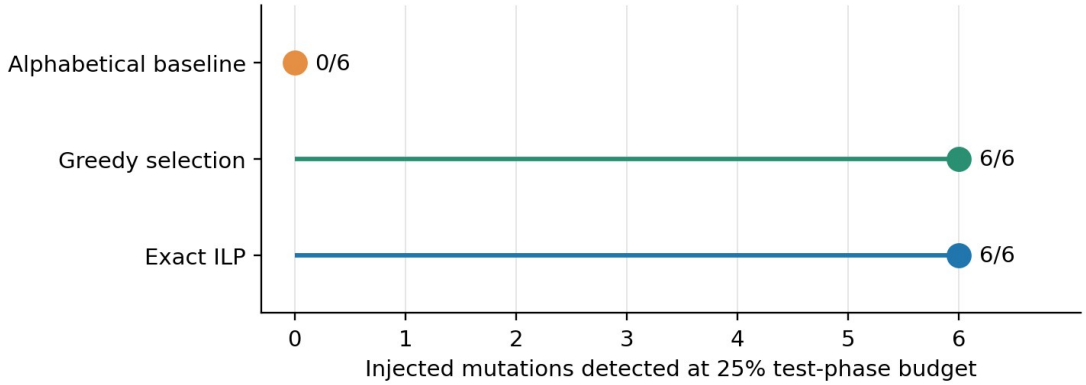


Figure 10. Fault-detection effectiveness of test-selection methods at the same declared test-phase budget.

Source and reproducibility: https://github.com/sofiaabidi/optimization-lab/tree/main/tracks/j_software_engineering; numerical artifacts in https://github.com/sofiaabidi/optimization-lab/tree/main/results/j.

13. Cross-track discussion

Several observations recur across the ten experiments. First, exact algorithms are most useful when they improve feasibility or objective value enough to justify their extra work. The EV charging LP identified allocations missed by the heuristic, whereas the SQLite ILP and test-selection ILP found no better solution than conditional greedy on their small measured instances. Second, a faster component does not necessarily improve a wider system. Extra processes introduced communication overhead, and the priority scheduler improved responsiveness while reducing throughput in one workload. Third, correctness must be checked independently from optimization claims: bytecode equivalence, query checksum equality, exact enumeration, and kernel usertests protect against a misleading “faster but wrong” result.

The experiments also differ in what performance numbers mean. Simulator gate or circuit counts should not be confused with hardware speedup; a QEMU xv6 timer tick is not directly comparable with a Python wall-clock microbenchmark; and an optimized objective on one synthetic problem is not a universal guarantee. Accordingly, percentages are discussed within their own controlled comparisons, not combined into a single cross-track score.

14. AI-assisted development and reproducibility

AI-assisted coding was used throughout the project, primarily through Codex. Implementation prompts, verification notes, AI-labelled commits and the AI tools register are retained in the repository. To assess AI output rather than treating it as automatically correct, the development

record includes three corrections and a controlled same-prompt comparison with a second coding tool.

14.1 AI-generated errors and corrections

Track C — Circuit gradient cancellation. An early gradient experiment used a circuit parameter whose effect cancelled, giving misleading gradient behavior. The parameterization was corrected and the revised experiment verified. The development record documents the correction; complete original failure output was not retained.

Track H — Throughput interpretation. The early database analysis conflated throughput calculated from latency with independently measured throughput. The reporting was corrected to distinguish the two measures, so the final throughput figures reflect separate measurements. Original raw failure output is limited.

Track I — CSV serialization. Heterogeneous experiment rows initially caused a CSV field mismatch and a serialization exception. The results writer was corrected and a regression test was added. This case includes a recorded exception and subsequent verification.

These cases were found and corrected during AI-assisted development. They illustrate why generated code and performance descriptions were checked against tests and measured artifacts, rather than accepted solely from an assistant response.

14.2 Same-task AI coding comparison

Codex (GPT-5.6 Medium, as selected in the comparison) and Antigravity (Claude Sonnet 4.6 Thinking) received an identical, self-contained request to implement 0/1 knapsack with dynamic programming, three tests, and complexity analysis. Neither had access to the optimization-lab repository for this comparison. Both produced bottom-up two-dimensional DP solutions with O(nW) time and O(nW) space and completed the initial response in one prompt iteration. Codex returned code directly in chat, whereas Antigravity created knapsack.py and reported that its three tests passed.

Both solutions were appropriate for ordinary positive-weight inputs. Codex explicitly checked capacity and item-weight validity. Antigravity had a zero-capacity early return that misses an allowed zero-weight, positive-value item: for items [(0, 5)] and capacity 0, the optimum is 5, but its implementation returns 0. No other material hallucination was identified from the submitted responses. Exact elapsed response times were not recorded, so neither tool is ranked by speed. The original responses and screenshots should be retained as authentic comparison evidence.

Comparison evidence: Codex and Antigravity original responses (to be archived in prompts/)

For reproducibility, the repository includes uv.lock, a Python 3.11 setup guide, commands to execute individual profiles, track-level result CSV/JSON files, and the xv6 patch plus boot/test

logs. All fast tests and artifact checks should be rerun as part of the final student submission check, and GitHub Actions should be confirmed on the final pushed commit. The student should retain dated screen captures or exported assistant conversations and a brief account of what was manually reviewed.

15. Conclusion

The ten experiments met the practical goal of learning optimization through measurement. The clearest gains came from choosing cheaper charging slots, selecting useful SQLite indexes, reducing compiler output, and redistributing skewed worker tasks. Equally valuable were results that did not improve: exact methods sometimes tied greedy, the compiler pass order optimized on training generalized poorly, and simulated quantum methods did not beat simple classical implementations on runtime or accuracy. The work supports an operational lesson: an optimization is only convincing when the objective is explicit, constraints remain satisfied, the comparison is fair, and its limitations are stated.

Appendix A — Submission and evidence notes

The final repository and report should be submitted together. The compulsory short demonstration can use Track E to illustrate input constraints, the baseline, the heuristic and LP result, with one before/after cost figure and a quick explanation of feasibility. The assigned report has no stated page limit; the layout is intended for normal university submission rather than a presentation deck.

Outstanding supporting evidence requiring student input: sanitized original assistant chat exports or screenshots, the two knapsack comparison responses with actual model selections, brief student review notes, and any raw original records available for the Track C and H correction cases. Quantum hardware runs and full-scale G–J profiles were not used for the reported findings.

Appendix B — Primary sources

Assignment specification: “Optimization Techniques | Lab Assignment 8 — Optimize Anything: The 14-Day Vibe-Coding Challenge,” supplied with the request. Repository: https://github.com/sofiaabidi/optimization-lab. For machine-readable data and reproducibility, see /results, /tracks and /docs/final_handoff.md in that repository. All numerical claims in this report are transcribed from the linked track reports and result summaries; figure values come from the same reported comparisons.